=> SpringREST => Restfullservices

⇓

pre-requisite

✓ ↳ Spring core + Spring Boot

✓ ↳ Spring MVC

↳ SpringREST

=> Java -> CUI/GUI standalone system }->

-> Web Application -> C to B

-> Distributed App ->

Enterprise App

=> Web Applications :-

=>We prefer using JEE module(Servlet,JSP) and Spring MVC to build these type of applications

=> The applications which runs over the internet is called "webapplications".

=>These application no need to download and no need to install. We just need to open browser software and access web application.

=> It uses client and server architecture.

=> client (browser) and in server computer we have project(App) From where service will be served to client.

Note: Main theme is Customer - Buisness communication.
example, Telusko web app, Facebook app, Gmail, ====.....

=> Distributed Application :-

↳ Webservices and RestFulservices

=> The application which is communicating with other applications is called as "Distrubuted applications".

C-B

App → App

B-B

Customer → Telusko => { paypall

courses

Course Access

Customer →

C — B

enduser
client / customer → Browser s/w → Telusko → paypal → Visa/mastercard

B-B   B-B   Distributed

{ camel Dashboard }

{ Telusko status }

HDFC, SBI, HSBC

C

=> { Telusko webApp }

=> weather forcast

B-B

C → Telusko → IBM

weather

=> SpringRest :- { Distributed Application }

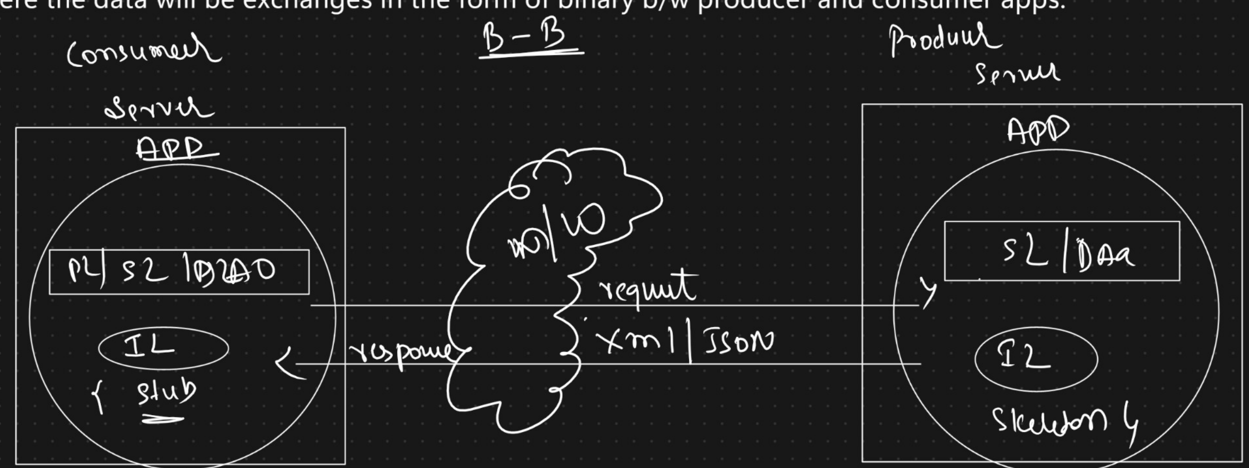
=>

{ a. RMI
b. EJB
c. CORBA }

{ d. webservices → SOAP =>
e. RESTful services → springREST =>

a. RMI

1. It stands for Remote Method Invocation, It is a part of JDK s/w only given by SUNMS(Oracle).
2. It doesnot support Transaction Management,Security,....
3. It is outdated because of EJB.
4. It is language dependent,platform dependent and Architecture dependent.
5. Here the data will be exchanges in the form of binary b/w producer and consumer apps.



b. EJB
1. It stands for Enterprise Java Beans.
2. It is given by SUNMS(Oracle)
3. Enhanced version of RMI
4.It is language dependent(coded only using java)
(communication of applications can happen only if the applications are coded in java only).
5. They were complex to learn and use.
6. Here Also the data will be exchanges in the form of binary b/w producer and consumer apps.

c. CORBA
1. It stands for Common Object Request Broker Architecture.
2. Complex to learn and use.
3. It is architecture and language independent.
4.Corba looks great conceptwise ,but gives problem towards the implementation.

{ JAX – RPC
JAX → WS }

⇒ App ↔ APP }   App ↔ App
Java   Java

{ APP
Java → { APP
. Python }
→ { APP
. Net } }

Inter-operability

Websirvices

8 to 10 → sun/oracle
8 to 10 → Microsoft
8 to 10 → C++

{ JAX – RPC }

→ Webservices Interoperability (WSI)
→ specification
↓
Distributed → inter oprasility
↓
{ B.P.1.0 }
↓
JAX-RPC
↓
(Java API for Xml – Remote procedural call)

⇒ { B. P.1.1 }
↓
{ JAX → WS → Java API for-xml @ web services } sums

{ Webservices
↳ APP ↔ APP ⇒ Xml
Java   Python
Interoprability

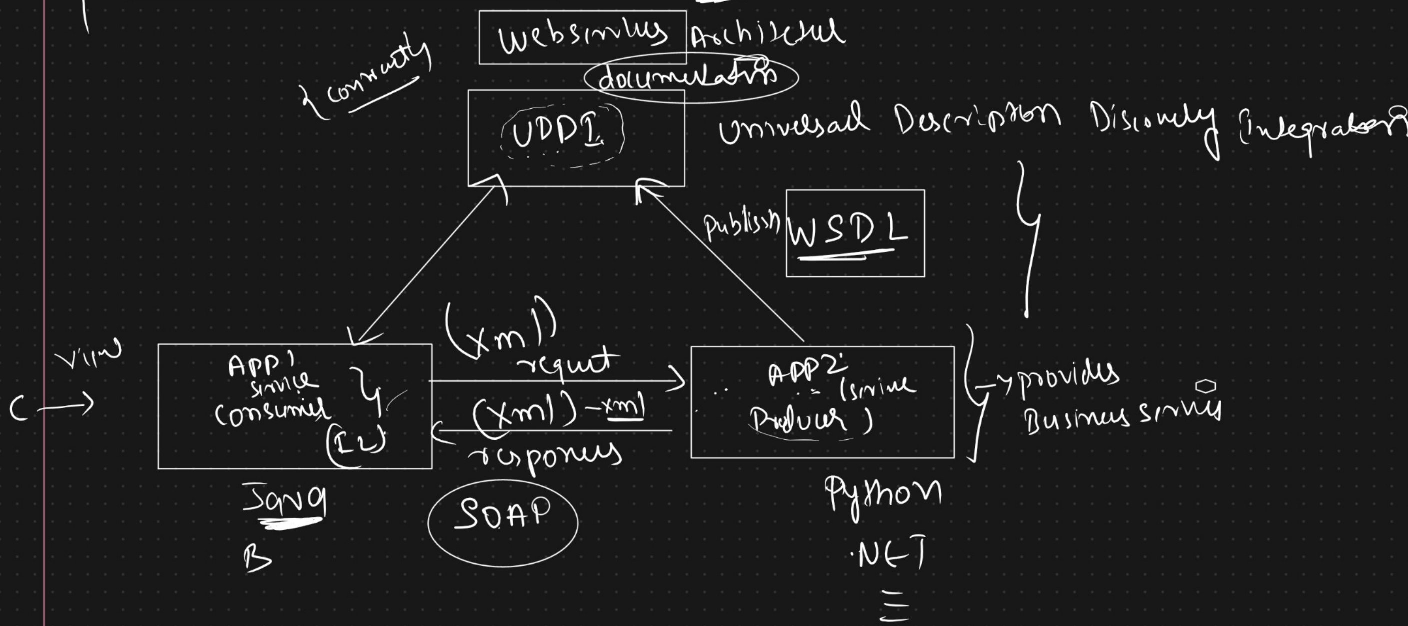
Java Developer

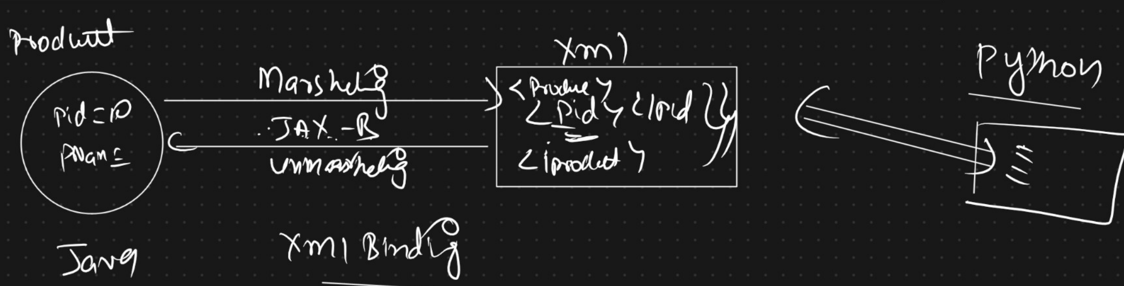
a. JAX-RPC -> xml => Sun MS -> Apache Axis, IBM websphere, oracle weblogic

b. JAX-WS -> Annotations

IBM websphere
oracle weblogic

SOAP -> Based webservices -> xml





==>To support Interopability Sun MicroSystem released one api called "JAX-RPC".
JAX-RPC(Java API for Xml - Remote Procedural Call) api is built on the specification of B.P.1.0(WSI organisation)

==> WSI -> WebServices Interoperability(ideas taken from many organisation and given the implementation)
B.P.1.0 is successfull,
WS-I released B.P.1.1 specification

==>W.r.t B.P.1.1 specification SUN Microsystem released JAX-WS api(Java api for XML - WebServices)

WebServices
===========
It is a distributed technology which is used to develop distributed applications with Interoperability.

What is Interoperablitiy?
Irrespective of the platform and the Irrespective of the programming language, if two applications are communicating then those applications are called as "Interoperable applications".

WebServices in java can be developed in 2 way
a. JAX-RPC(Java API for XML - Remote Procedure calls
b. JAX-WS(Java API For XML - WebServices)
==>If we develop a webservice using JAX-RPC or JAX-WS then that webservice is called as "SOAP Based WebService".

==>SOAP(Simple Object Access Protocol) WebServices is called as "Big Services".

In the world of webservices we have 2 parties

a. Provider => Application which will be providing buisness services to other applications is called as "Provider application".
b. Consumer => Application which is consuming the services from the other application is called as "Consumer application".

Contract?

=> Contract stands for WSDL
=> Web Services Description language
=> WSDL is a special XML which describes how provider is providing buisness services to consumers.

=> Contract First Approach means WSDL file will be created first then Development will be started.

=> Contract Last Approach means we will prepare the service first and then WSDL file will created.
WSDL -> url,input for provider,output for provider,how to access the services will be documented in the file.

==
==>Once provider developement is completed,provider will share WSDL share to consumer through email,sharepoint,UDDI(Universal Description Discovery Integration)

==>Once Consumer gets the WSDL file, we can start consumer development

===
Note:Key players when we build webservices
1. Provider
2. WSDL : WebServices Description Language
3. UDDI : Universal Discovery Description Language
4. Consumer
5. SOAP : Simple Object Access Protocol

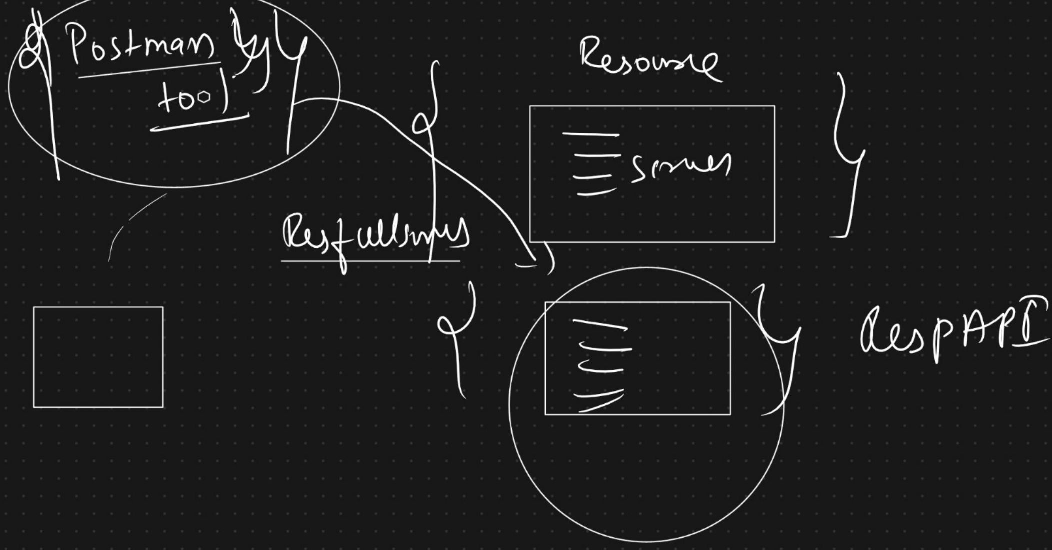
SOAP based webservices does not support for JSON/It supports only XML.
SOAP based webservices are not easily adoptable(dependent on XML).

The above mentioned problems are identified by a person called "Roy Fielding".
He compared SOAP webservices with Internet Services(www).

RoyFielding found 5 principles those are called as "REST Architecture principles".

REST Architecture
================
2 Actors are involved here

a. Resource ==>It provides buisness services to other applications.
b. Client ==> It access buisness services from other applications.



REST COMPONENTS
===============

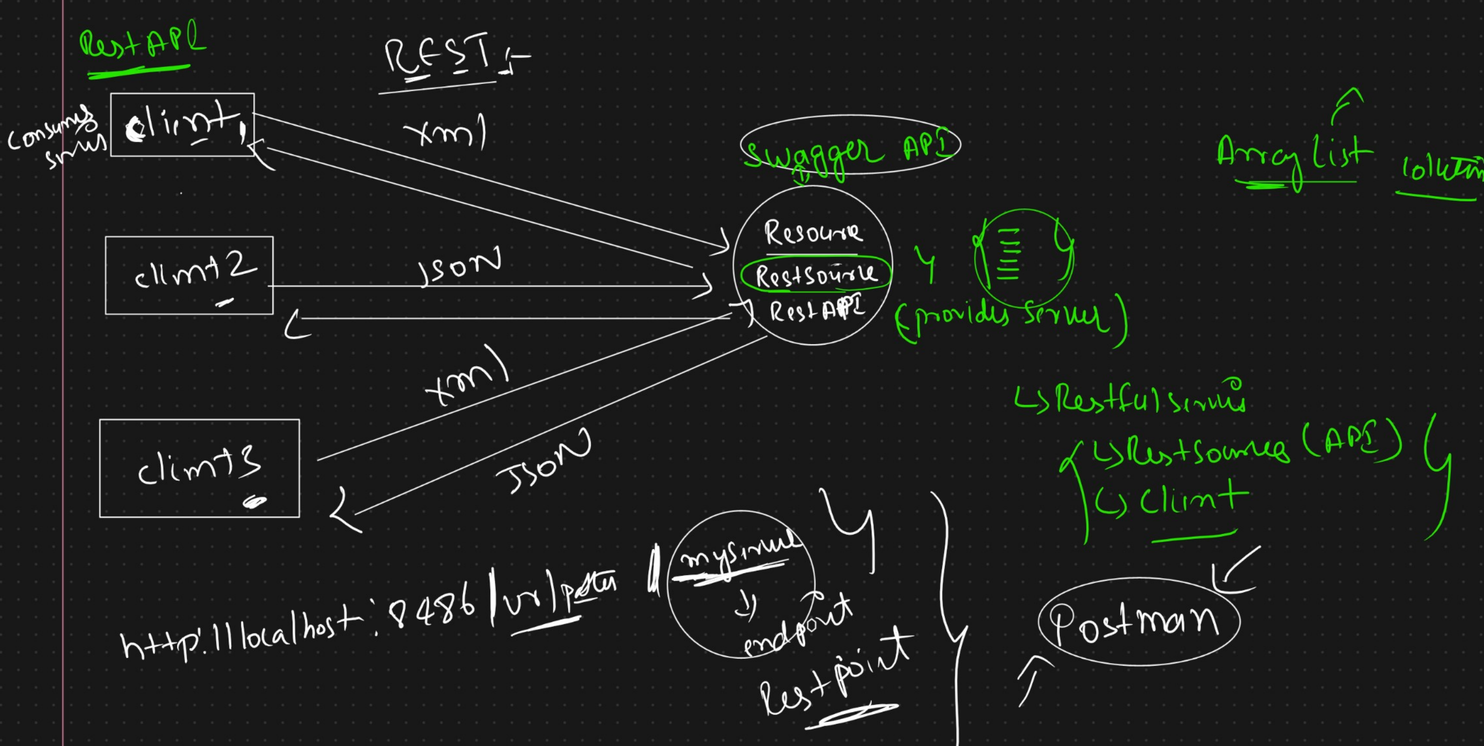
1. Resource(REST Resource)
2. WADL /Swagger
3. XML/JSON/
4. Client
5. PostMan(To test our application)

=> JSON/XML are language independent and platform independent.

RoyFielding identified few problems in B.P.1.1 specification, so SunMicroSystem released one api called "JAX-RS" api
JAX-RS (Java API For -Restful Services)

a. Jersey implementation(Sun)
b. Rest Easy implementation(JBoss)

Spring REST ==>
=> We can develop RestFulservices using Spring ReSt Module.
=> Spring MVC jars are sufficient to develop RestFul Services (additional jars are not required).



REST Architecture Principles
============================

1. Unique Address
2. Uniform Constraint interfaces
3. Media Representation
4. Communication Stateless
5. Hateos